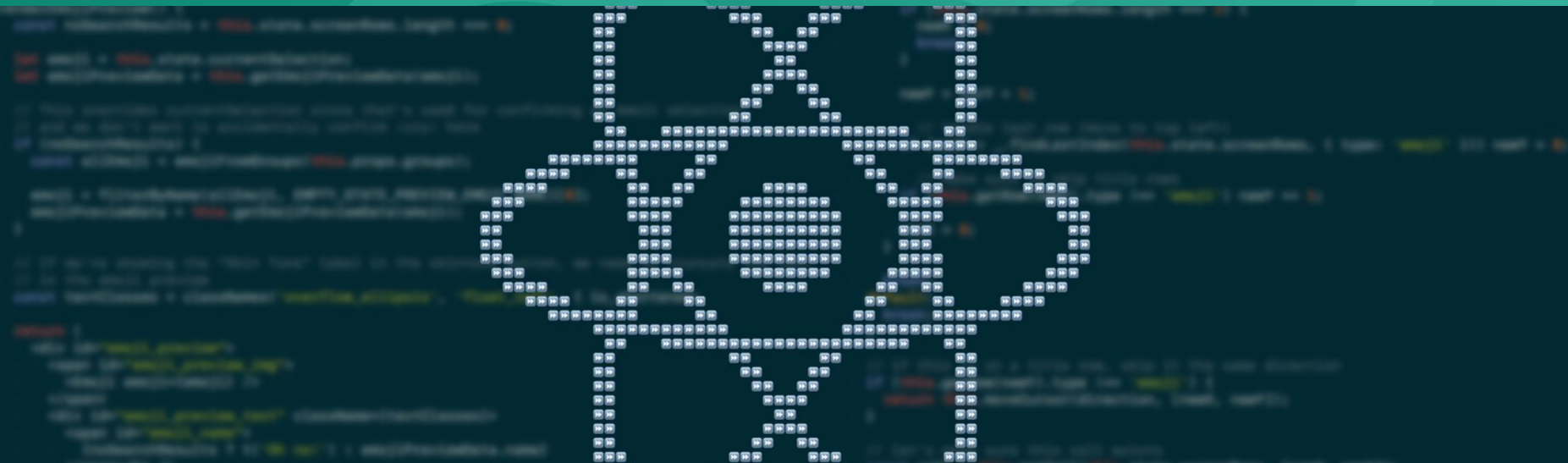


Crea componentes de clase

Clase 4





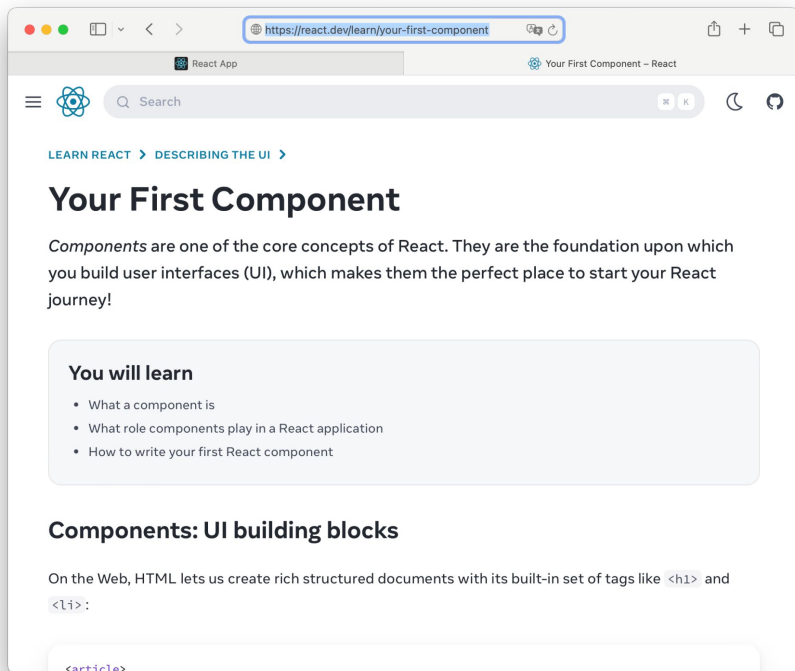
¿Qué son los componentes de clase?

Los componentes de clase son una forma de definir componentes en React utilizando la sintaxis de clase de JavaScript. En React, los componentes de clase extienden la clase base **'React.Component'** o alguna de sus subclases.

[Ver documentación](https://react.dev/learn/your-first-component)

Si no funciona el botón entra a:

<https://react.dev/learn/your-first-component>





create-react-app

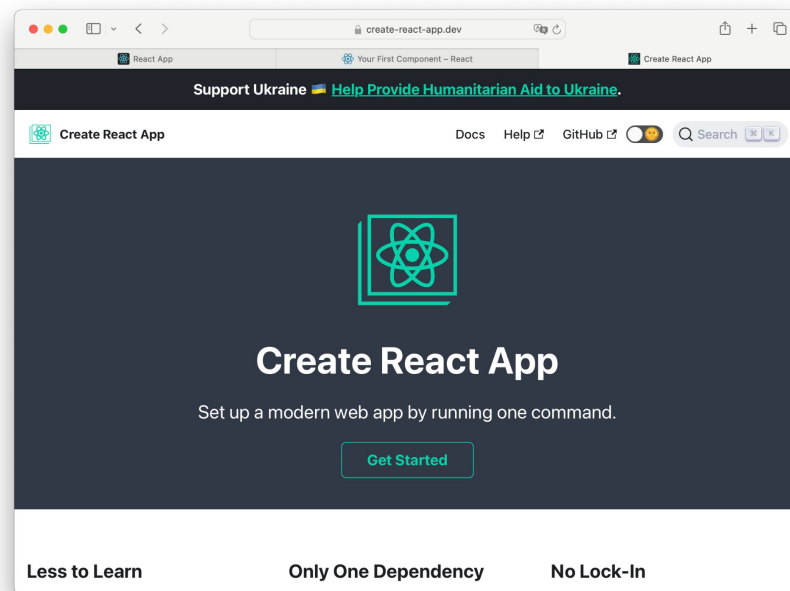
Como vimos, facilita la configuración inicial de un proyecto React al **proporcionar una estructura de directorios preconfigurada** y herramientas de desarrollo integradas.

Puedes **crear tus propios componentes de clase dentro del proyecto** generado por create-react-app. Por ejemplo, puedes definir un nuevo componente de clase MiComponente en un archivo como MiComponente.js.

Después de definir tus componentes de clase, **puedes importarlos y usarlos en la aplicación principal** (App.js por defecto). Puedes utilizar los componentes de clase como lo harías con cualquier otro componente en React.

Ver más

Si no funciona el botón entra a:
<https://create-react-app.dev>





Características de los Componentes de Clase

- 1) **Sintaxis de Clase:** Los componentes de clase se definen utilizando la sintaxis de clase de JavaScript. Esto permite utilizar características avanzadas de JavaScript como el estado local y los ciclos de vida del componente.
- 2) **Estado Local:** Los componentes de clase pueden tener estado local utilizando la propiedad **state**. El estado es un objeto que almacena datos que pueden cambiar durante el ciclo de vida del componente.
- 3) **Métodos de Ciclo de Vida:** Los componentes de clase tienen acceso a varios métodos de ciclo de vida que se ejecutan en diferentes etapas del ciclo de vida del componente, como `componentDidMount`, `componentDidUpdate` y `componentWillUnmount`.
- 4) **Renderización de JSX:** Los componentes de clase deben implementar un método llamado **render()** que devuelve el elemento JSX a renderizar. Este método es obligatorio en los componentes de clase y es donde se define la estructura de la interfaz de usuario del componente.

```
class Biology extends React.Component {
  constructor(props){
    super(props)
    this.state = {
      professor: "Franco Cuarterolo"
      students: props.numberOfStudents
    }
  }

  componentDidMount(){
    this.startAssignment()
  }

  componentWillUnmount(){
    this.finishAssignment()
  }

  render(){
    return(
      <div> Biology, Professor: {this.state.professor} </div>
    )
  }
}
```



Ejemplo de Componente de Clase

```
import React, { Component } from 'react';

class MiComponente extends Component {
  constructor(props) {
    super(props);
    this.state = {
      contador: 0
    };
  }

  componentDidMount() {
    console.log('El componente se montó');
  }

  handleClick = () => {
    this.setState(prevState => ({
      contador: prevState.contador + 1
    }));
  }

  render() {
    return (
      <div>
        <h1>Contador: {this.state.contador}</h1>
        <button onClick={this.handleClick}>Incrementar</button>
      </div>
    );
  }
}

export default MiComponente;
```

En este ejemplo:

- En este ejemplo, **MiComponente** es un componente de clase que define un estado local con un contador y un método **handleClick** para incrementar el contador.
- Se implementan los métodos de ciclo de vida **componentDidMount** y **render()** para mostrar el contador y un botón que permite incrementarlo.



Ejemplo de Componente de Clase

